

User space code

On a Linux system, the parallel port is a device node in the `/dev` folder with the name `parport0`. If you have more than one parallel port, then the other ports will be named `parport1`, `parport2`, ... `parportN`.

To use this parallel port from a user space program, we need to open the parallel port:

```
int fd;

fd=open("/dev/parport0",O_RDWR);
```

```
if(fd== -1)
{
    perror("open");
    exit(0);
}
```

So, let's try to open the port and if it fails, we print an error message and exit. Before controlling the port pins, we should first claim the port. Linux will not let us modify the port status unless we have successfully claimed it. Claiming the port requires just an `ioctl` call:

```
if(ioctl(fd,PPCLAIM) !=0)
{
    perror("ioctl");
    close(fd);
    exit(0);
}
```

If `ioctl` fails, that means some other program might have already claimed it or some other error has occurred, so we print the error message, close the `parport` that we have opened, and exit.

Just as we need to claim the port, we should also release it when we have finished working with the parallel port:

```
if(ioctl(fd,PPRELEASE) !=0)
    perror("ioctl");
```

But this release should be at the end of the program.

Now that we have claimed the port, controlling the pins again requires just some simple `ioctl` calls.

The `ioctl` commands that we will be using are:

1. `PPRCONTROL` – reads the control pins and gives us the data
2. `PPWCONTROL` – sets the control pins
3. `PPRDATA` – reads the data pins
4. `PPWDATA` – writes to the data pins

```
unsigned char r;
if(ioctl(fd,PPRCONTROL,&r)!=0)
{
    perror("ioctl");
```

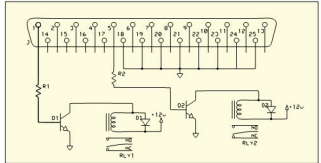


Figure 1: Schematic to connect devices

```
if(ioctl(fd,PPRELEASE) !=0)
    perror("ioctl");

close(fd);
exit(0);
}
```

We read the control lines and put the value in the variable 'r', as our device is connected to STROBE; so we need to set/reset bit 0 to control the STROBE pin.

```
unsigned int inp;

do {
    printf("Enter 1 to set, 0 to reset:\n");
    scanf("%u",&inp);
}while(inp>1); //stay in loop until user enters 0 or 1

r &= ~(0x01); //STROBE is first bit of control - reset the bit
r |= (!inp);
```

```
if(ioctl(fd,PPWCONTROL, &r) !=0)
    perror("ioctl");
```

After reading the control bits, we need to modify the bit 0 and again write that to the parallel port. All the other control pins will remain the same. Only our STROBE pin will change. `r &= ~(0x01)` will clear the first bit and `r |= (!inp)` will set/reset the first bit. We have used the '!' sign as STROBE is active-low, and when users enter 1, they will expect to see the pin go high.

If you want to add your devices to other control pins, then:

- Line-Feed is bit 1
- Reset is bit 2
- Select is bit 3

Similarly, to control the data pins, first read them, modify the particular pin and write again to the port.

```
if(ioctl(fd,PPRDATA,&r)!=0)
{
    perror("ioctl");
    if(ioctl(fd,PPRELEASE) !=0)
```